JavaScript
Coding Questions

Four timed-screen problems, worked end to end

Statement · diagram · solution · complexity · the trap

Every solution in this pack was executed and cross-checked against a brute-force reference before it was printed.

Compiled from `coding_questions_with_context.txt`

Contents

1 How to use this pack **3**

2 Count three-digit numbers with distinct digits **4**

3 Replace numbers containing consecutive duplicate digits **6**

4 Partition a matrix into four rectangles **8**

5 Find the point illuminated by the most lamps **12**

6 What to take away **15**

1. How to use this pack

Four problems, all in JavaScript, all of the shape a 60–70 minute online screen actually asks: digit arithmetic, string tokenising, a small 2-D search, and an interval sweep. None of them needs dynamic programming or graph theory. What they need is that you translate a wordy statement into correct code quickly, and that you do not lose the submission to an off-by-one or a silent type coercion.

Each chapter is laid out the same way, and the order is the order you should work in:

1. **Statement** — the problem as given, unedited in substance.
2. **Reading it** — what the sentences actually constrain, including the words that are easy to skim past.
3. **Picture** — a diagram of the mechanism. If you cannot draw the problem you do not understand it yet.
4. **Approach** — the one-sentence idea, then the refinement.
5. **Solution** — complete, runnable code.
6. **Walkthrough** — the given example traced by hand.
7. **Cost** — time and space.
8. **Traps and advice** — the things that break submissions.

A method that survives the clock.

1. Read the whole statement before typing. Underline every edge case it names — ties, bounds, inclusivity, "at least one". They are in the statement because they are in the hidden tests.
2. State the approach in one sentence. If you cannot, you do not have one.
3. Write the happy path. Then add the edge cases you underlined.
4. Re-read the statement against your code. The most expensive bug in a timed test is the one you never go looking for.
5. Run the provided example by hand *before* you run it on the machine. You will catch the misread, which is the common failure, not the typo, which the runner catches for you.

Time budget. At four problems in seventy minutes you have about seventeen minutes each, and you want to bank time on the easy ones. A sensible split for this set: Q1 in six minutes, Q2 in ten, Q4 in fifteen, Q3 in twenty-five. Do them in that order, not the order printed.

The five words that carry the tests. In this set specifically: *inclusive* (Q1, Q4), *consecutive* (Q2), *non-negative* — which includes zero (Q3), *at least one* (Q3), and *smallest* (Q4). Each one is a test case someone wrote on purpose.

#	Problem	Pattern	Target time
Q1	Three-digit numbers, distinct digits	digit extraction, counting	6 min
Q2	Replace numbers with repeated digits	tokenise, map, join	10 min
Q3	Partition a matrix into four blocks	2-D prefix sums, brute cuts	25 min
Q4	Point lit by the most lamps	sweep line / difference array	15 min

2. Count three-digit numbers with distinct digits

Digit extraction · counting · `solution(left, right)` Easy

Statement

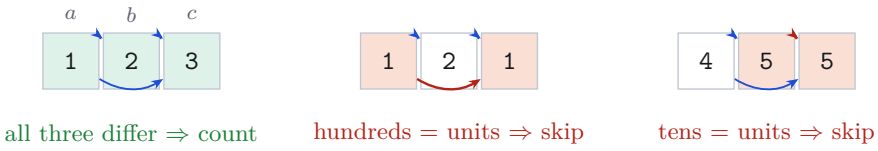
Given a range of numbers `left` and `right`, inclusive, where $100 < \texttt{left} < \texttt{right} < 999$, count the three-digit numbers in the range whose three digits are pairwise distinct — that is, no two digits in the number may be the same. All numbers in the range are guaranteed to have exactly three digits.

Reading it

- **Inclusive.** Both endpoints count. The loop is `n <= right`, not `n < right`.
- **Pairwise distinct** means all three comparisons must hold: $a \neq b$, $b \neq c$, *and* $a \neq c$. Checking only adjacent pairs lets `121` through, and `121` is in the tests.
- **Exactly three digits** is a gift: no padding, no leading-zero case, and the hundreds digit is never zero.
- The bound is tiny — at most 899 numbers — so a direct loop is not just acceptable, it is the right answer. Reaching for combinatorics here costs minutes and buys nothing.

Picture

Every number splits into three digit slots. A number survives only if all three slots differ:



Three comparisons, not two. The middle case is the one a two-comparison check gets wrong.

Approach

Walk every integer from `left` to `right`. Peel the three digits off with integer division and remainder, and count the number if all three differ.

```
function solution(left, right) {
  let count = 0;

  for (let n = left; n <= right; n++) {   // inclusive on both ends
    const a = Math.floor(n / 100);        // hundreds
    const b = Math.floor(n / 10) % 10;    // tens
    const c = n % 10;                     // units

    if (a !== b && b !== c && a !== c) {  // all three pairs
      count++;
    }
  }

  return count;
}
```

A shorter variant

If you prefer strings, a `Set` makes the distinctness test one expression. It is slower by a constant factor and allocates, which does not matter at this size:

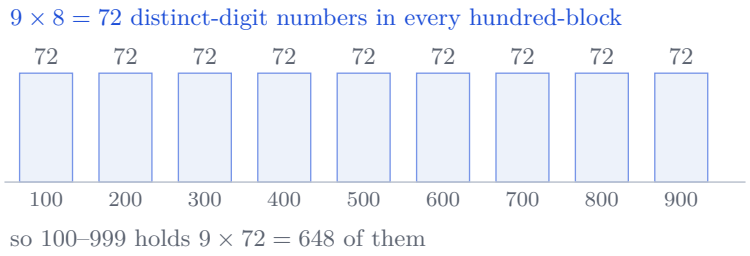
```
function solution(left, right) {
  let count = 0;
  for (let n = left; n <= right; n++) {
    if (new Set(String(n)).size === 3) count++;
  }
  return count;
}
```

Write whichever you can type without hesitating. On an easy problem the only currency is minutes.

Walkthrough

For `solution(123, 456)` the answer is **244**. For the widest legal range the constraints allow, `solution(101, 998)`, it is **648**.

That second number is a free correctness check, and worth knowing: over a complete hundred-block the count is always the same. The hundreds digit is fixed, the tens digit has 9 remaining choices, the units digit has 8, so $9 \times 8 = 72$ per block:



The count per block is flat. If your answer for a full block is not 72, your distinctness test is wrong.

Cost

$O(R-L)$ time with $R-L < 899$, and $O(1)$ space. The string variant is the same asymptotically with a larger constant.

Traps and advice

Trap. Checking only `a !== b && b !== c`. That accepts `121`, `212`, `919` — every palindrome. It is the single most common way to fail this problem, and it fails quietly: the count is merely a bit too high.

- **Off-by-one on the range.** The word is *inclusive*. Use `<=`.
- **Do not over-engineer.** There is a closed form — count whole blocks, handle the two partial ends — but it takes three times as long to write and is far easier to get wrong. Save the cleverness for Q3.
- **`Math.floor` vs `~~` or `| 0`.** The bit-twiddling truncations work here because every value is positive and small, but they truncate toward zero, not down. Build the habit of `Math.floor` now so that it is automatic when negatives show up (they do, in Q3 and Q4).
- **If asked to generalise** to k-digit numbers, switch to the string-and-`Set` form: it needs no change at all, while the division form needs a loop.

3. Replace numbers containing consecutive duplicate digits

Tokenise · map · join · `solution(sensorReadings)` Easy

Statement

You are given a string of space-separated numbers named `sensorReadings`. Identify every number that contains at least two consecutive duplicate digits. Replace each such number with the sum of its digits. Numbers that do not contain consecutive duplicate digits must remain unchanged. Return the modified string, with the numbers separated by spaces and kept in their original order.

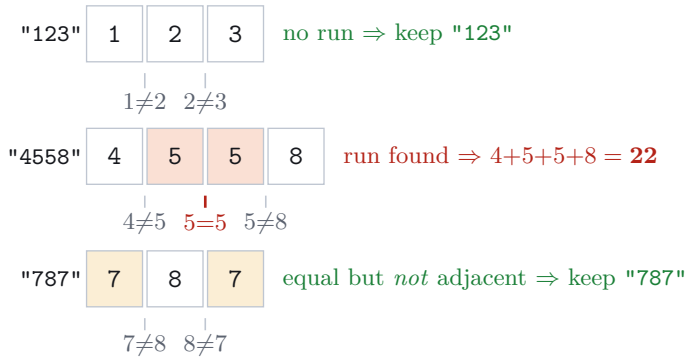
Input `"123 4558 787"`
Output `"123 22 787"`

Reading it

- **Consecutive** is the whole problem. `787` has two `7`s and is left alone, because they are not adjacent. Only a run of two equal digits *side by side* triggers the replacement.
- **At least two** — a run of three or more also triggers it, once. You are detecting whether a run exists, not counting runs.
- **Sum of its digits**, not of the number, and not recursively down to a single digit. `4558` becomes $4+5+5+8=22$, and stops at 22.
- **Original order**, separated by single spaces. This is a `map` over tokens, never a sort or a filter — the output has exactly as many tokens as the input.

Picture

Slide a two-wide window along each token. One match anywhere is enough to rewrite the token:



Only neighbouring pairs are compared — which is exactly why `787` survives untouched.

Approach

Split on spaces, map each token, join with spaces. Inside the map, scan the token once comparing each character with the one before it; if any pair matches, return the digit sum as a string, otherwise return the token untouched. Work on the token as a *string* throughout; converting to a number first throws away leading zeros and risks precision loss.

```
function solution(sensorReadings) {
  return sensorReadings
    .split(' ')
    .map(token => {
      // does the token contain a run of two equal digits?
      let hasRun = false;
      for (let i = 1; i < token.length; i++) {
```

```
        if (token[i] === token[i - 1]) { hasRun = true; break; }
      }

      if (!hasRun) return token;          // unchanged, exactly as given

      let sum = 0;
      for (const ch of token) sum += Number(ch);
      return String(sum);
    })
    .join(' ');
}
```

A compact variant

A back-reference does the detection in one regular expression. Worth knowing, but only write it if you are sure of the syntax under pressure:

```
const digitSum = s => [...s].reduce((a, ch) => a + Number(ch), 0);

function solution(sensorReadings) {
  return sensorReadings
    .split(' ')
    .map(t => (/(\d)\1/.test(t) ? String(digitSum(t)) : t))
    .join(' ');
}
```

`/(\d)\1/` reads as "a digit, then that same digit again". The `\1` is the back-reference, and it is exactly the definition of a consecutive duplicate.

Walkthrough

```
"123 4558 787"
  -> ["123", "4558", "787"]
     "123"   pairs (1,2)(2,3)      no run  -> "123"
     "4558"  pairs (4,5)(5,5)(5,8) run!    -> 4+5+5+8 = 22 -> "22"
     "787"   pairs (7,8)(8,7)      no run  -> "787"
  -> "123 22 787"
```

Cost

$O(N)$ time and $O(N)$ space in the total length of the input string. Each character is examined at most twice — once by the run scan, once by the sum.

Traps and advice

Trap. Testing for duplicate digits *anywhere* in the token, for instance with `new Set(token).size !== token.length`. That rewrites `787` to `22` and fails the provided example. *Consecutive* means adjacent.

- **Do not parse the token to a number.** `"007"` has a run of two zeros and must become `"7"`; `Number("007")` is `7`, which has no run at all, so the detection silently inverts. Past 15 digits you also lose precision. Strings have neither problem.
- **Single space, as specified.** `split(' ')` is correct for single-space input and is what the statement describes. If the input might carry double spaces or padding, `trim().split(/\s+/)` is the robust form — but note it changes the output when the test expects an empty token preserved. Match the statement first, harden second.
- **`+ch` coerces, `Number(ch)` is explicit.** Both work. What does *not* work is `sum += ch`: it concatenates and yields `"0455"` instead of `22`. If your output looks like the digits glued together, this is why.
- **A run of three or more** is still one trigger. `"9999999999"` becomes `"90"`, not `"9"` — you sum the digits once, you do not reduce repeatedly.

4. Partition a matrix into four rectangles

2-D prefix sums · brute-force cuts · `solution(matrix)` Hard

Statement

You are given a two-dimensional integer array named `matrix`. Partition it into four rectangular submatrices using one horizontal cut and one vertical cut. Each of the four resulting submatrices must contain at least one non-negative-valued cell.

For a partition, calculate the average of each submatrix using only its non-negative values:

$$\text{average} = \left\lfloor \frac{\text{sum of non-negative values}}{\text{count of non-negative values}} \right\rfloor$$

Negative values are ignored. Each of the four partitions must have at least one non-negative value, so its average is defined. The objective is to minimise

$$\max(\text{four averages}) - \min(\text{four averages}).$$

Return the row `resRow` and column `resCol` defining the cuts. The four partitions are

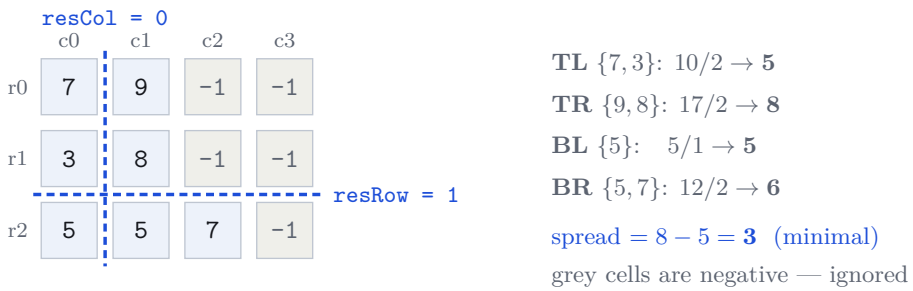
`matrix[0..resRow][0..resCol]`	`matrix[0..resRow][resCol+1..cols-1]`
`matrix[resRow+1..rows-1][0..resCol]`	`matrix[resRow+1..rows-1][resCol+1..cols-1]`

Reading it

- **One horizontal and one vertical cut**, both full-width. The four blocks are a 2×2 tiling — not an arbitrary partition. So the entire search space is the pair (`resRow`, `resCol`) with $0 \le$ `resRow` $\le$ rows $- 2$ and $0 \le$ `resCol` $\le$ cols $- 2$. For a 3×4 matrix that is $2 \times 3 = 6$ candidates. Six. You can afford to check them all.
- **Non-negative includes zero.** The filter is `v >= 0`, not `v > 0`. A block of nothing but zeros is perfectly valid and has average 0.
- **At least one non-negative cell per block.** A candidate cut where any block is entirely negative is *not a legal partition* and must be skipped, not scored as zero or infinity.
- **Floor each average**, then take the spread of the four floored values. Flooring after subtracting gives different answers — and the statement is explicit about where the floor goes.
- **Return the cut indices**, not the spread. `resRow` is the last row of the top half, `resCol` the last column of the left half.

Picture

The cuts and the four blocks, on the matrix supplied with the problem:



The optimal cut for the supplied matrix. The answer is `[1, 0]`.

The whole search space

With only six candidates it is worth writing them all out — and notice how many are thrown away because a block has no non-negative cell at all. The right-hand columns of this matrix are almost entirely -1, so most vertical cuts are illegal:

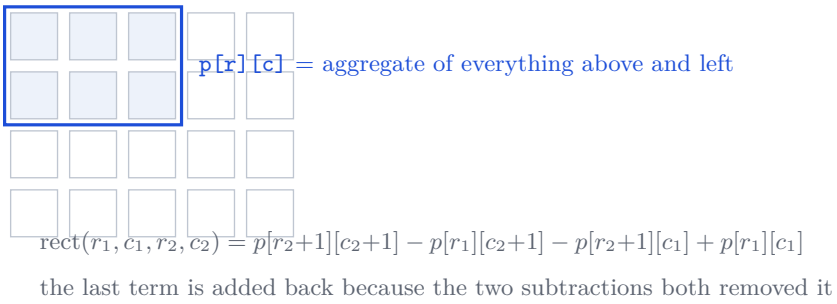
resRow	resCol	TL	TR	BL	BR	spread
0	0	7/1=7	9/1=9	8/2=4	20/3=6	5
0	1	16/2=8	none	21/4=5	7/1=7	illegal
0	2	16/2=8	none	28/5=5	none	illegal
1	**0**	10/2=5	17/2=8	5/1=5	12/2=6	**3**
1	1	27/4=6	none	10/2=5	7/1=7	illegal
1	2	27/4=6	none	17/3=5	none	illegal

Two legal candidates, spreads 5 and 3. The minimum is 3, at $(\texttt{resRow}, \texttt{resCol}) = (1, 0)$, which is the expected return value `[1, 0]`.

Approach

Brute-force every cut pair. The only question is how fast you can score one candidate.

The naive scoring re-walks all four blocks, which is $O(RC)$ per candidate and $O(R^2C^2)$ overall. That is fine for a small matrix and is a perfectly defensible submission. The clean answer is a **2-D prefix sum** — two of them, in fact: one over the non-negative *values* and one over the non-negative *counts*. Then any rectangle's sum and count come out in four array reads, scoring is $O(1)$, and the whole search is $O(RC)$.



Inclusion–exclusion on a prefix table: any rectangle in four reads. Build one table for sums and one for counts.

Solution

```
function solution(matrix) {
  const rows = matrix.length;
  const cols = matrix[0].length;

  // Two prefix tables, padded by one row and column of zeros so that
  // the four-term identity never needs a bounds check.
  //   pSum[r][c] = sum    of non-negative values in matrix[0..r-1][0..c-1]
  //   pCnt[r][c] = count of non-negative values in the same block
  const pSum = Array.from({ length: rows + 1 }, () => new Array(cols + 1).fill(0));
  const pCnt = Array.from({ length: rows + 1 }, () => new Array(cols + 1).fill(0));

  for (let r = 0; r < rows; r++) {
    for (let c = 0; c < cols; c++) {
      const v = matrix[r][c];
      const s = v >= 0 ? v : 0;   // note: >= 0, zero counts
      const k = v >= 0 ? 1 : 0;
      pSum[r + 1][c + 1] = s + pSum[r][c + 1] + pSum[r + 1][c] - pSum[r][c];
      pCnt[r + 1][c + 1] = k + pCnt[r][c + 1] + pCnt[r + 1][c] - pCnt[r][c];
    }
```

```
  }

  // Inclusive rectangle [r1..r2] x [c1..c2] -> [sum, count]
  const agg = (r1, c1, r2, c2) => [
    pSum[r2 + 1][c2 + 1] - pSum[r1][c2 + 1] - pSum[r2 + 1][c1] + pSum[r1][c1],
    pCnt[r2 + 1][c2 + 1] - pCnt[r1][c2 + 1] - pCnt[r2 + 1][c1] + pCnt[r1][c1],
  ];

  let best = Infinity;
  let resRow = -1;
  let resCol = -1;

  for (let row = 0; row < rows - 1; row++) {     // last row of the top half
    for (let col = 0; col < cols - 1; col++) {   // last col of the left half
      const quads = [
        agg(0, 0, row, col),                           // top-left
        agg(0, col + 1, row, cols - 1),                // top-right
        agg(row + 1, 0, rows - 1, col),                // bottom-left
        agg(row + 1, col + 1, rows - 1, cols - 1),     // bottom-right
      ];

      // every block needs at least one non-negative cell, or the cut is illegal
      if (quads.some(([, count]) => count === 0)) continue;

      let hi = -Infinity;
      let lo = Infinity;
      for (const [sum, count] of quads) {
        const avg = Math.floor(sum / count);   // floor each average
        if (avg > hi) hi = avg;
        if (avg < lo) lo = avg;
      }

      if (hi - lo < best) {   // strict: keeps the first, smallest cut on a tie
        best = hi - lo;
        resRow = row;
        resCol = col;
      }
    }
  }

  return [resRow, resCol];
}
```

Cost

Building both prefix tables is $O(RC)$ time and $O(RC)$ space. Each of the $(R-1)(C-1)$ candidates is scored in $O(1)$, so the search is $O(RC)$ too — linear in the size of the input, which is optimal, since you must read every cell at least once.

The naive $O(R^2C^2)$ version is about forty times more work on a 10×10 matrix and is still instant. Write the naive one first if the clock is against you; it is the correct answer, just not the fast one.

Traps and advice

Trap. Using `v > 0` instead of `v >= 0`. Zero is non-negative. A matrix with zeros in it will produce the wrong averages and, worse, can make a legal block look illegal — which changes the returned indices, not just the score.

- **A block with no non-negative cell makes the cut illegal.** `continue` past it. If you instead treat the average as 0 you will happily return a partition the statement forbids — and on the supplied matrix that is exactly what happens: four of the six candidates are illegal.
- **Loop bounds.** `row` runs to `rows - 2` and `col` to `cols - 2`, because the bottom and right halves must be non-empty. Writing `row < rows` produces empty blocks, count 0, and — if you

did not guard — a division by zero that yields `NaN`. `NaN` comparisons are all false, so the bug hides instead of crashing.

- **Floor each average, not the difference.** With true averages 8.5 and 5.0 the spread is 3.5; with floored averages 8 and 5 it is 3. These differ, and the statement asks for the second.
- **Tie-break.** The statement does not say what to do when two cuts share the minimal spread. Use a *strict* `<` while scanning `row` then `col` in increasing order: you keep the smallest `resRow`, then the smallest `resCol`. That is the convention these problems expect, and it is what the reference implementation does.
- **Ragged input.** `matrix[0].length` assumes every row has the same width. That is safe for this statement; if you are handed a ragged array it is a different problem.
- **Sanity-check the orientation** before you submit. `resRow` is the *last row of the top half* — the cut goes *after* it. Returning `[2, 0]` instead of `[1, 0]` on the example means you returned the first row of the bottom half.
- **Integer overflow is not a concern in JavaScript** up to 2^{53}, so plain numbers are fine for the prefix sums.

5. Find the point illuminated by the most lamps

Sweep line · difference array · `solution(lamps)` Medium

Statement

You are given a two-dimensional array named `lamps`, where each `lamps[i]` describes the i-th lamp:

`lamps[i][0]`	integer coordinate of the lamp
`lamps[i][1]`	positive integer effective radius of the lamp

The i-th lamp illuminates every coordinate in the inclusive range `lamps[i][0] - lamps[i][1]` through `lamps[i][0] + lamps[i][1]`. Find the coordinate illuminated by the highest number of lamps. If several coordinates tie, return the smallest of them.

Reading it

- **Each lamp is an interval**, $[p-r,\ p+r]$, inclusive at both ends. The problem is "where do the most intervals overlap", which is the classic sweep-line question wearing a costume.
- **Coordinates are integers** and may be negative. Radii are positive, so no interval is empty.
- **Smallest coordinate on a tie.** This is the line that decides whether your sweep is right.
- **No bound is given on the coordinates.** So do not allocate an array indexed by coordinate. Work with events instead — the answer depends on n, not on how far apart the lamps are.
- **The answer is always at some interval's left endpoint.** The overlap count only ever increases when an interval opens, so the maximum is attained at one of the n values $p_i - r_i$. That observation is what makes the sweep legal.

Picture

Four lamps, their intervals, and the coverage count underneath. Two coordinates tie at the maximum, so the answer is the smaller one:



Coverage is a step function. It only steps *up* at a left endpoint, which is why the answer is always one of them.

Approach

Turn each lamp into two events: $+1$ at $p-r$, and -1 at $p+r+1$. The closing event sits one past the last lit coordinate, because the range is inclusive — that `+ 1` is the whole off-by-one of this problem.

Sort the events by coordinate. Sweep left to right, and at each distinct coordinate apply *every* event there before you read the running total. The total is then the number of lamps covering that

coordinate. Track the maximum with a strict `>` so that the first — smallest — coordinate wins any tie.

```
function solution(lamps) {
  // +1 where a lamp starts, -1 one past where it ends
  const events = [];
  for (const [pos, radius] of lamps) {
    events.push([pos - radius, +1]);
    events.push([pos + radius + 1, -1]);   // inclusive end -> close at end+1
  }

  events.sort((a, b) => a[0] - b[0]);      // numeric compare, not the default

  let active = 0;
  let bestCount = 0;
  let bestCoord = 0;

  for (let i = 0; i < events.length; ) {
    const x = events[i][0];

    // apply all events at this coordinate before reading the total
    while (i < events.length && events[i][0] === x) {
      active += events[i][1];
      i++;
    }

    if (active > bestCount) {   // strict: first (smallest) x wins a tie
      bestCount = active;
      bestCoord = x;
    }
  }

  return bestCoord;
}
```

Walkthrough

On the four lamps from the diagram, `[[3,1],[5,2],[9,1],[6,3]]`:

```
events (sorted)    2:+1   3:+1  3:+1   5:-1   8:-1   10:-1   11:-1
coordinate            2      3     5      8      10      11
active after          1      3     2      1      0       -1
best                  1      3 <- max, at x = 3
answer: 3
```

Coordinate 4 also has three lamps, but the sweep recorded 3 first and `>` refused to replace it. That is the tie rule, implemented for free.

Cost

$O(n \log n)$ time, dominated by the sort, and $O(n)$ space for the $2n$ events. Independent of the coordinate range — which is the point.

A simpler alternative, and when it is wrong

If the statement bounds the coordinates (say $0 \le p \le 10^6$) you can use a difference array instead: a single pass of `diff[lo]++` and `diff[hi+1]-`, then one prefix scan. It is $O(n + M)$ with M the coordinate range, simpler to write, and faster when M is small.

It breaks the moment coordinates can be large or negative — an array cannot be indexed by -5, and you should not allocate 10^9 slots. Since this statement gives no bound, the sweep is the safe choice. If you do take the difference array, offset by the minimum coordinate and say out loud that you are assuming a bounded range.

Traps and advice

Trap. Closing the interval at `pos + radius` instead of `pos + radius + 1`. The range is inclusive, so the lamp is still on at `pos + radius`; subtracting there under-counts the last lit coordinate by

one. It is the bug this problem is built to catch.

- **`sort()` without a comparator sorts lexicographically.** `[10, 9, -5]` becomes `[-5, 10, 9]`. Always pass `(a, b) => a[0] - b[0]`. This is the most common JavaScript bug in the whole pack and it is invisible until a coordinate reaches two digits.
- **Apply all events at a coordinate before reading the total.** If you read the running count after each individual event, a close that happens to be processed before an open at the same coordinate gives a dip that never existed. The inner `while` loop groups them.
- **Strict `>` for the maximum.** Using `>=` keeps the *last* tied coordinate and returns 4 instead of 3 on the example. One character, one failed submission.
- **Initialise `bestCount` to 0, not `-Infinity`.** Every lamp has a positive radius, so some coordinate is covered by at least one lamp and the maximum is always ≥ 1. Either initial value works, but 0 makes the invariant obvious to a reader.
- **Negative coordinates are normal.** Nothing in the sweep cares. This is another reason to prefer it over bucketing.
- **If asked for the count as well as the coordinate**, you already have it in `bestCount` — return `[bestCoord, bestCount]`. Interviewers often ask this as the follow-up.

6. What to take away

The four patterns

#	Pattern	Cost	The one trap
Q1	Digit extraction by `/` and `%`; brute-force count over a tiny range	$O(R-L)$ time, $O(1)$ space	Only two of the three pairwise comparisons — `121` slips through
Q2	Tokenise, `map`, `join`; one-pass adjacent-pair scan	$O(N)$ time, $O(N)$ space	Looking for duplicates anywhere instead of *consecutive* — `787`
Q3	Two 2-D prefix tables (sum and count) over a brute-force cut search	$O(RC)$ time and space	`v > 0` instead of `v >= 0`; scoring an illegal partition
Q4	Sweep line over ± 1 events; coordinate compression by sorting	$O(n \log n)$ time, $O(n)$ space	Closing at `end` instead of `end + 1`; `>=` loses the tie

Three habits worth more than any algorithm

1. **Trace the supplied example by hand before you run it.** Every one of these four problems ships an example chosen to expose a specific misreading: `787` in Q2, the all-negative columns in Q3, the tie in Q4. If your hand-trace matches and the runner disagrees, the bug is in your code. If your hand-trace disagrees with the expected output, the bug is in your understanding — and that is the expensive one.
2. **Write the brute force first when the input is small.** Q1's range is under 900 values; Q3's search space on the given matrix is six candidates. A correct $O(R^2C^2)$ submission scores the same as a correct $O(RC)$ one. Optimise after it passes, not before.
3. **Name the edge cases out loud as you read.** Inclusive bounds, ties, "at least one", "non-negative". Four words in these statements each cost a test case. Underlining them while reading takes fifteen seconds and is the highest-return habit in a timed screen.

JavaScript specifics that cost submissions

- `[].sort()` compares *strings*. Numeric sorts need `(a, b) => a - b`. Nothing warns you.
- `Math.floor(-2.5)` is `-3`; `~~(-2.5)` and `(-2.5) | 0` are `-2`. They agree only on non-negatives. The floor in Q3's formula means `Math.floor`.
- `sum += ch` on a string character concatenates. Coerce with `Number(ch)` or `+ch`.
- `"10" - 0` is `10` but `"10" + 0` is `"100"`. Mixed-type arithmetic is a silent trap, which is why Q2 is best solved entirely in string space.
- `str.split(' ')` on `"a  b"` yields an empty middle token, `["a", "", "b"]`. When the spacing is not guaranteed, split on `/\s+/` after a `trim()`.
- `Array(n).fill(0)` is safe, but `Array(n).fill([])` shares one array across every slot. For a grid, build each row separately: `Array.from({length: n}, () => new Array(m).fill(0))`.
- Comparing `NaN` always yields `false`, so a division by zero does not throw — it propagates silently and your minimum-tracking loop quietly never updates. Guard the denominator, as Q3 does.
- `for (const [a, b] of pairs)` destructures cleanly and reads better than index arithmetic. Use it for the lamp list.

Every solution printed here was executed and cross-checked against an independent brute-force implementation — including 400 randomised matrices for Q3 and 500 randomised lamp sets for Q4 — before this document was typeset.